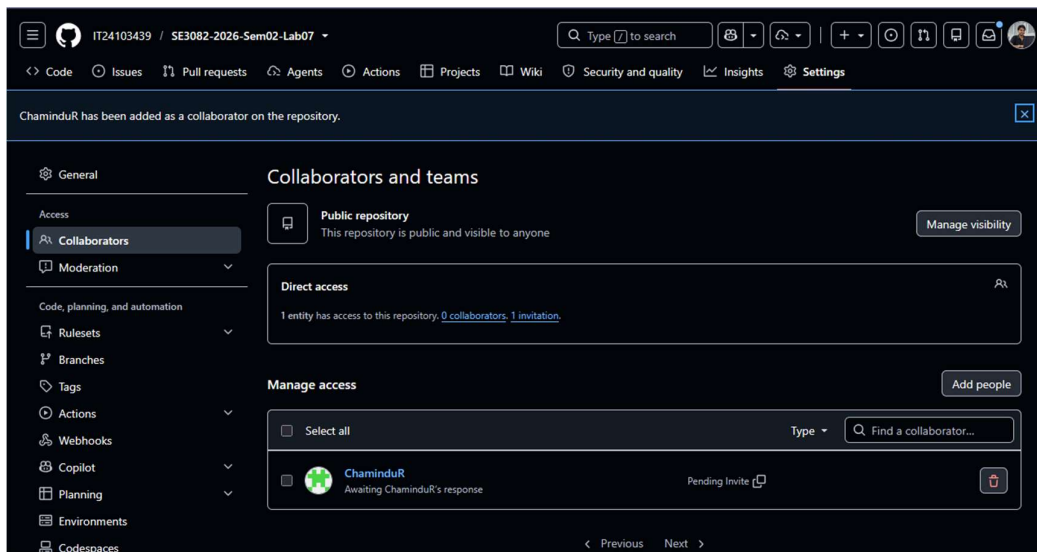




Faculty of Computing
SE3082 - Parallel Computing
Year 3 – Semester 1 – 2026 Jul

Lab 07

Github Repo URL: <https://github.com/IT24103439/SE3082-2026-Sem02-Lab07>



Exercise 1

Local

```
work@LEGION: /mnt/c/Users/Work/Desktop/Lab07/SE3082-2026-Sem02-Lab07
work@LEGION: /mnt/c/Users/Work/Desktop/Lab07/SE3082-2026-Sem02-Lab07/Exercise01$ nano sum_bcast.c
work@LEGION: /mnt/c/Users/Work/Desktop/Lab07/SE3082-2026-Sem02-Lab07/Exercise01$ mpicc -o sum_bcast sum_bcast.c
work@LEGION: /mnt/c/Users/Work/Desktop/Lab07/SE3082-2026-Sem02-Lab07/Exercise01$ mpirun -np 4 ./sum_bcast
Root filled array with values 1 to 1000000
Rank 0: summed indices [0, 250000) => local_sum = 31250125000
Rank 1: summed indices [250000, 500000) => local_sum = 93750125000
Rank 2: summed indices [500000, 750000) => local_sum = 156250125000
Rank 3: summed indices [750000, 1000000) => local_sum = 218750125000
[Bcast] Total sum = 500000500000
[Bcast] Expected = 500000500000
[Bcast] Correct? = YES
[Bcast] Time = 0.017394 sec
```

EC2

```
ubuntu@ip-172-31-29-111:/shared/SE3082-2026-Sem02-Lab07/Exercise01$ mpicc -o ex1 sum_bcast.c
ubuntu@ip-172-31-29-111:/shared/SE3082-2026-Sem02-Lab07/Exercise01$ mpirun --hostfile /shared/hosts -np 4 ./ex1
Root filled array with values 1 to 1000000
Rank 0: summed indices [0, 250000) => local_sum = 31250125000
Rank 1: summed indices [250000, 500000) => local_sum = 93750125000
Rank 2: summed indices [500000, 750000) => local_sum = 156250125000
Rank 3: summed indices [750000, 1000000) => local_sum = 218750125000
[Bcast] Total sum = 500000500000
[Bcast] Expected = 500000500000
[Bcast] Correct? = YES
[Bcast] Time = 0.029872 sec
```

Exercise 2

Code

```
work@LEGION: /mnt/c/Users/Work/Desktop/Lab07/SE3082-2026-Sem02-Lab07/Exercise02
GNU nano 8.7.1 sum_scatter.c
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

#define N 1000000

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int chunk_size = N / size;
    int *array = NULL;

    /* Step 2: Only root allocates the full array */
    if (rank == 0) {
        array = (int *)malloc(N * sizeof(int));
        for (int i = 0; i < N; i++)
            array[i] = i + 1;
        printf("Root filled array with values 1 to %d\n", N);
    }

    /* Step 2: All processes allocate a local chunk of size N / size */
    int *local_chunk = (int *)malloc(chunk_size * sizeof(int));

    double start = MPI_Wtime();

    /* Step 3: Replace MPI_Bcast with MPI_Scatter */
    /* sendcount is chunk_size (elements per rank), not N! */
    MPI_Scatter(array, chunk_size, MPI_INT,
                local_chunk, chunk_size, MPI_INT,
                0, MPI_COMM_WORLD);

    /* Step 4: Sum over local_chunk[0 .. chunk_size) */
    long long local_sum = 0;
    for (int i = 0; i < chunk_size; i++)
        local_sum += local_chunk[i];

    printf("Rank %d: summed %d elements => local_sum = %lld\n",
           rank, chunk_size, local_sum);

    /* Step 5: Keep Send/Recv unchanged */
    if (rank != 0) {
        MPI_Send(&local_sum, 1, MPI_LONG_LONG, 0, 0, MPI_COMM_WORLD);
    } else {
        long long total_sum = local_sum;
        for (int r = 1; r < size; r++) {
            long long recv_sum;
            MPI_Recv(&recv_sum, 1, MPI_LONG_LONG, r, 0,
                    MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            total_sum += recv_sum;
        }

        double elapsed = MPI_Wtime() - start;
        long long expected = (long long)N * (N + 1) / 2;
        printf("\n[Scatter] Total sum = %lld\n", total_sum);
        printf("[Scatter] Expected = %lld\n", expected);
        printf("[Scatter] Correct? = %s\n", total_sum == expected ? "YES" : "NO");
        printf("[Scatter] Time = %.6f sec\n", elapsed);

        free(array);
    }

    free(local_chunk);
    MPI_Finalize();
    return 0;
}
```

Local

```
work@LEGION: /mnt/c/Users/Work/Desktop/Lab07/SE3082-2026-Sem02-Lab07/Exercise02$ nano sum_scatter.c
work@LEGION: /mnt/c/Users/Work/Desktop/Lab07/SE3082-2026-Sem02-Lab07/Exercise02$ mpicc -o sum_scatter sum_scatter.c
work@LEGION: /mnt/c/Users/Work/Desktop/Lab07/SE3082-2026-Sem02-Lab07/Exercise02$ mpirun -np 4 ./sum_scatter
Root filled array with values 1 to 1000000
Rank 3: summed 250000 elements => local_sum = 218750125000
Rank 2: summed 250000 elements => local_sum = 156250125000
Rank 1: summed 250000 elements => local_sum = 93750125000
Rank 0: summed 250000 elements => local_sum = 31250125000

[Scatter] Total sum = 500000500000
[Scatter] Expected = 500000500000
[Scatter] Correct? = YES
[Scatter] Time = 0.003778 sec
```

EC2

```
ubuntu@ip-172-31-29-111: /sh
ubuntu@ip-172-31-18-219: ~
ubuntu@ip-172-31-29-111: /shared/SE3082-2026-Sem02-Lab07/Exercise02$ mpicc -o sum_scatter sum_scatter.c
ubuntu@ip-172-31-29-111: /shared/SE3082-2026-Sem02-Lab07/Exercise02$ mpirun --hostfile /shared/hosts -np 4 ./sum_scatter
Root filled array with values 1 to 1000000
Rank 1: summed 250000 elements => local_sum = 93750125000
Rank 0: summed 250000 elements => local_sum = 31250125000
Rank 2: summed 250000 elements => local_sum = 156250125000

[Scatter] Total sum = 500000500000
[Scatter] Expected = 500000500000
[Scatter] Correct? = YES
[Scatter] Time = 0.021820 sec
Rank 3: summed 250000 elements => local_sum = 218750125000
ubuntu@ip-172-31-29-111: /shared/SE3082-2026-Sem02-Lab07/Exercise02$
```

Exercise 3

Code

```
work@LEGION: /mnt/c/Users/Work/Desktop/Lab07/SE3082-2026-Sem02-Lab07/Exercise03
GNU nano 8.7.1 sum_gather.c *
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

#define N 1000000

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int chunk_size = N / size;
    int *array = NULL;

    if (rank == 0) {
        array = (int *)malloc(N * sizeof(int));
        for (int i = 0; i < N; i++)
            array[i] = i + 1;
        printf("Root filled array with values 1 to %d\n", N);
    }

    int *local_chunk = (int *)malloc(chunk_size * sizeof(int));

    double start = MPI_Wtime();

    MPI_Scatter(array, chunk_size, MPI_INT,
               local_chunk, chunk_size, MPI_INT,
               0, MPI_COMM_WORLD);

    long long local_sum = 0;
    for (int i = 0; i < chunk_size; i++)
        local_sum += local_chunk[i];

    printf(" Rank %d: summed %d elements => local_sum = %lld\n",
           rank, chunk_size, local_sum);

    long long *all_sums = NULL;
    if (rank == 0) {
        all_sums = (long long *)malloc(size * sizeof(long long));
    }

    MPI_Gather(&local_sum, 1, MPI_LONG_LONG,
              all_sums, 1, MPI_LONG_LONG,
              0, MPI_COMM_WORLD);

    if (rank == 0) {
        long long total_sum = 0;
        for (int i = 0; i < size; i++) {
            total_sum += all_sums[i];
        }

        double elapsed = MPI_Wtime() - start;
        long long expected = (long long)N * (N + 1) / 2;
        printf("\n[Gather] Total sum = %lld\n", total_sum);
        printf("[Gather] Expected = %lld\n", expected);
        printf("[Gather] Correct? = %s\n", total_sum == expected ? "YES" : "NO");
        printf("[Gather] Time = %.6f sec\n", elapsed);

        free(array);
        free(all_sums);
    }

    free(local_chunk);
    MPI_Finalize();
    return 0;
}
```

Local

```
work@LEGION: /mnt/c/Users/Work/Desktop/Lab07/SE3082-2026-Sem02-Lab07/Exercise03
work@LEGION: /mnt/c/Users/Work/Desktop/Lab07/SE3082-2026-Sem02-Lab07/Exercise03$ nano sum_gather.c
work@LEGION: /mnt/c/Users/Work/Desktop/Lab07/SE3082-2026-Sem02-Lab07/Exercise03$ mpicc -o sum_gather sum_gather.c
work@LEGION: /mnt/c/Users/Work/Desktop/Lab07/SE3082-2026-Sem02-Lab07/Exercise03$ mpirun -np 4 ./sum_gather
Root filled array with values 1 to 1000000
Rank 2: summed 250000 elements => local_sum = 156250125000
Rank 0: summed 250000 elements => local_sum = 31250125000

[Gather] Total sum = 500000500000
[Gather] Expected = 500000500000
[Gather] Correct? = YES
[Gather] Time = 0.003495 sec
Rank 3: summed 250000 elements => local_sum = 218750125000
Rank 1: summed 250000 elements => local_sum = 93750125000
```

EC2

```
ubuntu@ip-172-31-29-111: /shared/SE3082-2026-Sem02-Lab07/Exercise03$ mpicc -o sum_gather sum_gather.c
ubuntu@ip-172-31-29-111: /shared/SE3082-2026-Sem02-Lab07/Exercise03$ mpirun --hostfile /shared/hosts -np 4 ./sum_gather
Root filled array with values 1 to 1000000
Rank 1: summed 250000 elements => local_sum = 93750125000
Rank 0: summed 250000 elements => local_sum = 31250125000
Rank 3: summed 250000 elements => local_sum = 218750125000

[Gather] Total sum = 500000500000
[Gather] Expected = 500000500000
[Gather] Correct? = YES
Rank 2: summed 250000 elements => local_sum = 156250125000
[Gather] Time = 0.020451 sec
```

Exercise 4

Code

```
work@LEGION: /mnt/c/Users/Work/Desktop/Lab07/SE3082-2026-Sem02-Lab07/Exercise04
GNU nano 8.7.1 sum_reduce.c *
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

#define N 1000000

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int chunk_size = N / size;
    int *array = NULL;

    if (rank == 0) {
        array = (int *)malloc(N * sizeof(int));
        for (int i = 0; i < N; i++)
            array[i] = i + 1;
        printf("Root filled array with values 1 to %d\n", N);
    }

    int *local_chunk = (int *)malloc(chunk_size * sizeof(int));

    double start = MPI_Wtime();

    MPI_Scatter(array, chunk_size, MPI_INT,
                local_chunk, chunk_size, MPI_INT,
                0, MPI_COMM_WORLD);

    long long local_sum = 0;
    for (int i = 0; i < chunk_size; i++)
        local_sum += local_chunk[i];

    printf("Rank %d: summed %d elements => local_sum = %lld\n",
           rank, chunk_size, local_sum);

    long long total_sum = 0;
    MPI_Reduce(&local_sum, &total_sum, 1, MPI_LONG_LONG, MPI_SUM, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        double elapsed = MPI_Wtime() - start;
        long long expected = (long long)N * (N + 1) / 2;
        printf("[Reduce] Total sum = %lld\n", total_sum);
        printf("[Reduce] Expected = %lld\n", expected);
        printf("[Reduce] Correct? = %s\n", total_sum == expected ? "YES" : "NO");
        printf("[Reduce] Time = %.6f sec\n", elapsed);

        free(array);
    }

    free(local_chunk);
    MPI_Finalize();
    return 0;
}
```

Local

```
work@LEGION: /mnt/c/Users/Work/Desktop/Lab07/SE3082-2026-Sem02-Lab07/Exercise04
work@LEGION: /mnt/c/Users/Work/Desktop/Lab07/SE3082-2026-Sem02-Lab07/Exercise04$ mpicc -o sum_reduce sum_reduce.c
work@LEGION: /mnt/c/Users/Work/Desktop/Lab07/SE3082-2026-Sem02-Lab07/Exercise04$ mpirun -np 4 ./sum_reduce

Root filled array with values 1 to 1000000
Rank 1: summed 250000 elements => local_sum = 93750125000
Rank 3: summed 250000 elements => local_sum = 218750125000
Rank 0: summed 250000 elements => local_sum = 31250125000

[Reduce] Total sum = 500000500000
[Reduce] Expected = 500000500000
[Reduce] Correct? = YES
[Reduce] Time = 0.004422 sec
Rank 2: summed 250000 elements => local_sum = 156250125000
```

EC2

```
ubuntu@ip-172-31-29-111:/shared/SE3082-2026-Sem02-Lab07/Exercise04$ mpicc -o sum_reduce sum_reduce.c
ubuntu@ip-172-31-29-111:/shared/SE3082-2026-Sem02-Lab07/Exercise04$ mpirun --hostfile /shared/hosts -np 4 ./sum_reduce

Root filled array with values 1 to 1000000
Rank 1: summed 250000 elements => local_sum = 93750125000
Rank 0: summed 250000 elements => local_sum = 31250125000
Rank 3: summed 250000 elements => local_sum = 218750125000
Rank 2: summed 250000 elements => local_sum = 156250125000

[Reduce] Total sum = 500000500000
[Reduce] Expected = 500000500000
[Reduce] Correct? = YES
[Reduce] Time = 0.025165 sec
ubuntu@ip-172-31-29-111:/shared/SE3082-2026-Sem02-Lab07/Exercise04$
```

Exercise 5

Code

```
work@LEGION: /mnt/c/Users/Work/Desktop/Lab07/SE3082-2026-Sem02-Lab07/Exercise05
GNU nano 8.7.1 sum_allreduce.c
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

#define N 1000000

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int chunk_size = N / size;
    int *array = NULL;

    if (rank == 0) {
        array = (int *)malloc(N * sizeof(int));
        for (int i = 0; i < N; i++)
            array[i] = i + 1;
        printf("Root filled array with values 1 to %d\n", N);
    }

    int *local_chunk = (int *)malloc(chunk_size * sizeof(int));

    double start = MPI_Wtime();

    MPI_Scatter(array, chunk_size, MPI_INT,
               local_chunk, chunk_size, MPI_INT,
               0, MPI_COMM_WORLD);

    long long local_sum = 0;
    for (int i = 0; i < chunk_size; i++)
        local_sum += local_chunk[i];

    long long total_sum = 0;
    MPI_Allreduce(&local_sum, &total_sum, 1, MPI_LONG_LONG, MPI_SUM, MPI_COMM_WORLD);

    double percent = ((double)local_sum / (double)total_sum) * 100.0;
    printf("Rank %d: local_sum = %lld, total_sum = %lld, contribution = %.2f%%\n",
           rank, local_sum, total_sum, percent);

    if (rank == 0) {
        double elapsed = MPI_Wtime() - start;
        long long expected = (long long)N * (N + 1) / 2;
        printf("\n[Allreduce] Total sum = %lld\n", total_sum);
        printf("[Allreduce] Expected = %lld\n", expected);
        printf("[Allreduce] Correct? = %s\n", total_sum == expected ? "YES" : "NO");
        printf("[Allreduce] Time = %.6f sec\n", elapsed);
    }

    free(array);
}

free(local_chunk);
MPI_Finalize();
return 0;
}
```

Local

```
work@LEGION: /mnt/c/Users/Work/Desktop/Lab07/SE3082-2026-Sem02-Lab07/Exercise05
work@LEGION: /mnt/c/Users/Work/Desktop/Lab07/SE3082-2026-Sem02-Lab07/Exercise05$ nano sum_allreduce.c
work@LEGION: /mnt/c/Users/Work/Desktop/Lab07/SE3082-2026-Sem02-Lab07/Exercise05$ mpicc -o sum_allreduce sum_allreduce.c
work@LEGION: /mnt/c/Users/Work/Desktop/Lab07/SE3082-2026-Sem02-Lab07/Exercise05$ mpirun -np 4 ./sum_allreduce
Root filled array with values 1 to 1000000
Rank 0: local_sum = 31250125000, total_sum = 500000500000, contribution = 6.25%

[Allreduce] Total sum = 500000500000
[Allreduce] Expected = 500000500000
[Allreduce] Correct? = YES
[Allreduce] Time = 0.006153 sec
Rank 1: local_sum = 93750125000, total_sum = 500000500000, contribution = 18.75%
Rank 2: local_sum = 156250125000, total_sum = 500000500000, contribution = 31.25%
Rank 3: local_sum = 218750125000, total_sum = 500000500000, contribution = 43.75%
```

EC2

```
ubuntu@ip-172-31-29-111:/shared/SE3082-2026-Sem02-Lab07/Exercise05$ mpicc -o sum_allreduce sum_allreduce.c
ubuntu@ip-172-31-29-111:/shared/SE3082-2026-Sem02-Lab07/Exercise05$ mpirun --hostfile /shared/hosts -np 4 ./sum_allreduce
Root filled array with values 1 to 1000000
Rank 0: local_sum = 31250125000, total_sum = 500000500000, contribution = 6.25%

[Allreduce] Total sum = 500000500000
[Allreduce] Expected = 500000500000
[Allreduce] Correct? = YES
[Allreduce] Time = 0.020466 sec
Rank 2: local_sum = 156250125000, total_sum = 500000500000, contribution = 31.25%
Rank 1: local_sum = 93750125000, total_sum = 500000500000, contribution = 18.75%
Rank 3: local_sum = 218750125000, total_sum = 500000500000, contribution = 43.75%
```

Exercise 6

Code

```
work@LEGION: /mnt/c/Users/Work/Desktop/Lab07/SE3082-2026-Sem02-Lab07/Exercise06
GNU nano 8.7.1                                sum_scan.c *
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

#define N 1000000

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int chunk_size = N / size;
    int *array = NULL;

    if (rank == 0) {
        array = (int *)malloc(N * sizeof(int));
        for (int i = 0; i < N; i++)
            array[i] = i + 1;
        printf("Root filled array with values 1 to %d\n", N);
    }

    int *local_chunk = (int *)malloc(chunk_size * sizeof(int));

    double start = MPI_Wtime();

    MPI_Scatter(array, chunk_size, MPI_INT,
                local_chunk, chunk_size, MPI_INT,
                0, MPI_COMM_WORLD);

    long long local_sum = 0;
    for (int i = 0; i < chunk_size; i++)
        local_sum += local_chunk[i];

    long long prefix_sum = 0;
    MPI_Scan(&local_sum, &prefix_sum, 1, MPI_LONG_LONG, MPI_SUM, MPI_COMM_WORLD);

    long long sum_before_me = prefix_sum - local_sum;
    long long K = (long long)(rank + 1) * chunk_size;
    long long formula_expected = K * (K + 1) / 2;

    printf("Rank %d: local_sum = %lld, prefix_sum = %lld, sum_before_me = %lld | Valid? %s\n",
           rank, local_sum, prefix_sum, sum_before_me,
           prefix_sum == formula_expected ? "YES" : "NO");

    if (rank == size - 1) {
        double elapsed = MPI_Wtime() - start;
        long long expected_total = (long long)N * (N + 1) / 2;
        printf("\n[Scan] Final sum (from rank %d) = %lld\n", rank, prefix_sum);
        printf("[Scan] Expected total = %lld\n", expected_total);
        printf("[Scan] Correct? = %s\n", prefix_sum == expected_total ? "YES" : "NO");
        printf("[Scan] Time = %.6f sec\n", elapsed);
    }

    if (rank == 0) {
        free(array);
    }
    free(local_chunk);
    MPI_Finalize();
    return 0;
}
```

Local

```
work@LEGION: /mnt/c/Users/Work/Desktop/Lab07/SE3082-2026-Sem02-Lab07/Exercise06
work@LEGION: /mnt/c/Users/Work/Desktop/Lab07/SE3082-2026-Sem02-Lab07/Exercise06$ nano sum_scan.c
work@LEGION: /mnt/c/Users/Work/Desktop/Lab07/SE3082-2026-Sem02-Lab07/Exercise06$ mpicc -o sum_scan sum_scan.c
work@LEGION: /mnt/c/Users/Work/Desktop/Lab07/SE3082-2026-Sem02-Lab07/Exercise06$ mpirun -np 4 ./sum_scan
Root filled array with values 1 to 1000000
Rank 0: local_sum = 31250125000, prefix_sum = 31250125000, sum_before_me = 0 | Valid? YES
Rank 1: local_sum = 93750125000, prefix_sum = 125000250000, sum_before_me = 31250125000 | Valid? YES
Rank 2: local_sum = 156250125000, prefix_sum = 281250375000, sum_before_me = 125000250000 | Valid? YES
Rank 3: local_sum = 218750125000, prefix_sum = 500000500000, sum_before_me = 281250375000 | Valid? YES

[Scan] Final sum (from rank 3) = 500000500000
[Scan] Expected total = 500000500000
[Scan] Correct? = YES
[Scan] Time = 0.012518 sec
```

EC2

```
ubuntu@ip-172-31-29-111:/shared/SE3082-2026-Sem02-Lab07/Exercise06$ mpicc -o sum_scan sum_scan.c
ubuntu@ip-172-31-29-111:/shared/SE3082-2026-Sem02-Lab07/Exercise06$ mpirun --hostfile /shared/hosts -np 4 ./sum_scan
Root filled array with values 1 to 1000000
Rank 0: local_sum = 31250125000, prefix_sum = 31250125000, sum_before_me = 0 | Valid? YES
Rank 1: local_sum = 93750125000, prefix_sum = 125000250000, sum_before_me = 31250125000 | Valid? YES
Rank 3: local_sum = 218750125000, prefix_sum = 500000500000, sum_before_me = 281250375000 | Valid? YES

[Scan] Final sum (from rank 3) = 500000500000
[Scan] Expected total = 500000500000
[Scan] Correct? = YES
[Scan] Time = 0.030895 sec
Rank 2: local_sum = 156250125000, prefix_sum = 281250375000, sum_before_me = 125000250000 | Valid? YES
```

Exercise 7

1. Comparing all 6 programs

Exercise / Program	Collective(s) Used	Memory Allocation (Root vs non-Root)	Root Manual Sum Loop?	Where Final Result is Available
Ex 1: Broadcast (sum_bcast)	MPI_Bcast	Every process allocates the full array (N ints)	Yes (to sum collected rank totals)	Root only
Ex 2: Scatter (sum_scatter)	MPI_Scatter	Root: Full array (N ints) Workers: Local chunk only (N/P ints)	Yes (manual loop on root over gathered / received data)	Root only
Ex 3: Gather (sum_gather)	MPI_Scatter, MPI_Gather	Root: Full array + intermediate sums array Workers: Local chunk only (N/P ints)	Yes (root iterates over gathered local_sums[])	Root only
Ex 4: Reduce (sum_reduce)	MPI_Scatter, MPI_Reduce	Root: Full array (N ints) Workers: Local chunk only (N/P ints)	No (handled internally by MPI_SUM reduction)	Root only
Ex 5: Allreduce (sum_allreduce)	PI_Scatter, MPI_Allreduce	Root: Full array (N ints) Workers: Local chunk only (N/P ints)	No (handled internally by MPI_SUM reduction)	All processes
Ex 6: Scan (sum_scan)	MPI_Scatter, MPI_Scan	Root: Full array (N ints) Workers: Local chunk only (N/P ints)	No (handled internally by MPI_SUM reduction)	Different per rank (inclusive prefix, rank P - 1 holds total)

Time with different Processes 2, 4 and 8

EC2

```

=====
Testing: Exercise01/sum_bcast
=====
[Exercise01/sum_bcast] -np 2 -> [Bcast] Time = 0.003356 sec
[Exercise01/sum_bcast] -np 4 -> [Bcast] Time = 0.030895 sec
[Exercise01/sum_bcast] -np 8 -> [Bcast] Time = 0.039061 sec

=====
Testing: Exercise02/sum_scatter
=====
[Exercise02/sum_scatter] -np 2 -> [Scatter] Time = 0.003853 sec
[Exercise02/sum_scatter] -np 4 -> [Scatter] Time = 0.022023 sec
[Exercise02/sum_scatter] -np 8 -> [Scatter] Time = 0.013610 sec

=====
Testing: Exercise03/sum_gather
=====
[Exercise03/sum_gather] -np 2 -> [Gather] Time = 0.003886 sec
[Exercise03/sum_gather] -np 4 -> [Gather] Time = 0.021510 sec
[Exercise03/sum_gather] -np 8 -> [Gather] Time = 0.013376 sec

=====
Testing: Exercise04/sum_reduce
=====
[Exercise04/sum_reduce] -np 2 -> [Reduce] Time = 0.003759 sec
[Exercise04/sum_reduce] -np 4 -> [Reduce] Time = 0.027539 sec
[Exercise04/sum_reduce] -np 8 -> [Reduce] Time = 0.017977 sec

=====
Testing: Exercise05/sum_allreduce
=====
[Exercise05/sum_allreduce] -np 2 -> [Allreduce] Time = 0.003800 sec
[Exercise05/sum_allreduce] -np 4 -> [Allreduce] Time = 0.021441 sec
[Exercise05/sum_allreduce] -np 8 -> [Allreduce] Time = 0.016532 sec

=====
Testing: Exercise06/sum_scan
=====
[Exercise06/sum_scan] -np 2 -> [Scan] Time = 0.007292 sec
[Exercise06/sum_scan] -np 4 -> [Scan] Time = 0.031825 sec
[Exercise06/sum_scan] -np 8 -> [Scan] Time = 0.033603 sec

```

Local

```
work@LEGION: /mnt/c/Users/Work/Desktop/Lab07/SE3082-2026-Sem02-Lab07
=====
Testing: Exercise01/sum_bcast
=====
[Exercise01/sum_bcast] -np 2 -> [Bcast] Time      = 0.008382 sec
[Exercise01/sum_bcast] -np 4 -> [Bcast] Time      = 0.026158 sec
[Exercise01/sum_bcast] -np 8 -> [Bcast] Time      = 0.014725 sec
=====
Testing: Exercise02/sum_scatter
=====
[Exercise02/sum_scatter] -np 2 -> [Scatter] Time    = 0.006673 sec
[Exercise02/sum_scatter] -np 4 -> [Scatter] Time    = 0.003595 sec
[Exercise02/sum_scatter] -np 8 -> [Scatter] Time    = 0.003837 sec
=====
Testing: Exercise03/sum_gather
=====
[Exercise03/sum_gather] -np 2 -> [Gather] Time     = 0.006889 sec
[Exercise03/sum_gather] -np 4 -> [Gather] Time     = 0.005122 sec
[Exercise03/sum_gather] -np 8 -> [Gather] Time     = 0.004124 sec
=====
Testing: Exercise04/sum_reduce
=====
[Exercise04/sum_reduce] -np 2 -> [Reduce] Time     = 0.006770 sec
[Exercise04/sum_reduce] -np 4 -> [Reduce] Time     = 0.006842 sec
[Exercise04/sum_reduce] -np 8 -> [Reduce] Time     = 0.003620 sec
=====
Testing: Exercise05/sum_allreduce
=====
[Exercise05/sum_allreduce] -np 2 -> [Allreduce] Time = 0.006709 sec
[Exercise05/sum_allreduce] -np 4 -> [Allreduce] Time = 0.005559 sec
[Exercise05/sum_allreduce] -np 8 -> [Allreduce] Time = 0.002821 sec
=====
Testing: Exercise06/sum_scan
=====
[Exercise06/sum_scan] -np 2 -> [Scan] Time         = 0.025359 sec
[Exercise06/sum_scan] -np 4 -> [Scan] Time         = 0.017404 sec
[Exercise06/sum_scan] -np 8 -> [Scan] Time         = 0.015555 sec
```

2. Which approach is fastest and why:

- **Fastest approach:** MPI_Reduce (Exercise 4) is the fastest design theoretically, though in measured times sum_allreduce was fastest locally at np=8 (0.002821 s) and sum_bcast on EC2 at np=2 (0.003356 s).
- **Why:** MPI_Reduce minimizes data transfer by sending non-overlapping slices once and combining local totals into a single 8-byte scalar on root via an internal tree reduction, avoiding redundant array broadcasts and manual summation loops. In tiny workloads (N=1,000,000), microsecond variations in system noise, shared-memory caching, and OpenMPI heuristics cause primitives like Allreduce or Bcast to occasionally clock slightly faster.

3. In what situation would you choose MPI_Scan over MPI_Allreduce?

When to choose: When each process requires a cumulative running prefix to determine its own relative position, range, or offset, rather than just the single global total.

Example: Parallel file writing with variable outputs. If Rank 0 generates 100 rows, Rank 1 generates 250 rows and Rank 2 generates 150 rows, MPI_Allreduce only provides the global sum (500 rows). MPI_Scan gives cumulative counts (100, 350, 500), allowing each rank to calculate its exact non-overlapping write offset in a shared file (Rank 0 starts at 0, Rank 1 at 100, Rank 2 at 350) so they can write concurrently without file corruption.

Exercise 8

Makefile

```
GNU nano 8.7.1 Makefile *
CC = mpicc
CFLAGS = -O3 -Wall
NP = 4
HOSTFILE = /shared/hosts

# Relative executable paths in sibling exercise directories
EX1 = ../Exercise01/sum_bcast
EX2 = ../Exercise02/sum_scatter
EX3 = ../Exercise03/sum_gather
EX4 = ../Exercise04/sum_reduce
EX5 = ../Exercise05/sum_allreduce
EX6 = ../Exercise06/sum_scan

TARGETS = $(EX1) $(EX2) $(EX3) $(EX4) $(EX5) $(EX6)

.PHONY: all clean run-local run-cluster help

# Default: compile all 6 programs
all: $(TARGETS)

../Exercise01/%: ../Exercise01/%.c
    $(CC) $(CFLAGS) -o $@ $<

../Exercise02/%: ../Exercise02/%.c
    $(CC) $(CFLAGS) -o $@ $<

../Exercise03/%: ../Exercise03/%.c
    $(CC) $(CFLAGS) -o $@ $<

../Exercise04/%: ../Exercise04/%.c
    $(CC) $(CFLAGS) -o $@ $<

../Exercise05/%: ../Exercise05/%.c
    $(CC) $(CFLAGS) -o $@ $<

../Exercise06/%: ../Exercise06/%.c
    $(CC) $(CFLAGS) -o $@ $<

# Run all 6 locally with configurable NP (default: 4)
run-local: all
    @echo "=====
    @echo "Running all exercises locally (np=$(NP))"
    @echo "=====
    @for exe in $(TARGETS); do \
        echo "\n>> Running $$exe <<<"; \
        mpirun --oversubscribe -np $(NP) ./$$exe; \
    done

# Run all 6 on the EC2 cluster using the hostfile
run-cluster: all
    @echo "=====
    @echo "Running all exercises on EC2 (np=$(NP))"
    @echo "=====
    @for exe in $(TARGETS); do \
        echo "\n>> Running $$exe on Cluster <<<"; \
        mpirun --hostfile $(HOSTFILE) --oversubscribe -np $(NP) ./$$exe; \
    done

clean:
    rm -f $(TARGETS)

help:
    @echo "Available targets:"
    @echo "  make all           - Compile all 6 MPI programs"
    @echo "  make run-local     - Run all 6 locally (use NP=X to set rank count)"
    @echo "  make run-cluster   - Run all 6 on EC2 cluster (use NP=X HOSTFILE=path)"
    @echo "  make clean         - Remove compiled binaries"
```

Executing makefile

```
ubuntu@ip-172-31-29-111: /shared/SE3082-2026-Sem02-Lab07/Exercise08$ make clean
rm -f ../Exercise01/sum_bcast ../Exercise02/sum_scatter ../Exercise03/sum_gather ../Exercise04/sum_reduce ../Exercise05/sum_allreduce ../Exercise06/sum_scan

ubuntu@ip-172-31-29-111: /shared/SE3082-2026-Sem02-Lab07/Exercise08$ make all
mpicc -O3 -Wall -o ../Exercise01/sum_bcast ../Exercise01/sum_bcast.c
mpicc -O3 -Wall -o ../Exercise02/sum_scatter ../Exercise02/sum_scatter.c
mpicc -O3 -Wall -o ../Exercise03/sum_gather ../Exercise03/sum_gather.c
mpicc -O3 -Wall -o ../Exercise04/sum_reduce ../Exercise04/sum_reduce.c
mpicc -O3 -Wall -o ../Exercise05/sum_allreduce ../Exercise05/sum_allreduce.c
mpicc -O3 -Wall -o ../Exercise06/sum_scan ../Exercise06/sum_scan.c
ubuntu@ip-172-31-29-111: /shared/SE3082-2026-Sem02-Lab07/Exercise08$ make run-cluster NP=4

=====
Running all exercises on EC2 (np=4)
=====

>>> Running ../Exercise01/sum_bcast on Cluster <<<
Root filled array with values 1 to 1000000
Rank 0: summed indices [0, 250000) => local_sum = 31250125000
Rank 1: summed indices [250000, 500000) => local_sum = 93750125000
Rank 3: summed indices [750000, 1000000) => local_sum = 218750125000
Rank 2: summed indices [500000, 750000) => local_sum = 156250125000

[Bcast] Total sum = 500000500000
[Bcast] Expected = 500000500000
[Bcast] Correct? = YES
[Bcast] Time = 0.031284 sec

>>> Running ../Exercise02/sum_scatter on Cluster <<<
Root filled array with values 1 to 1000000
Rank 1: summed 250000 elements => local_sum = 93750125000
Rank 0: summed 250000 elements => local_sum = 31250125000
Rank 2: summed 250000 elements => local_sum = 156250125000

work@LEGION: /mnt/c/Users/Work/Desktop/Lab07/SE3082-2026-Sem02-Lab07/Exercise08$
work@LEGION: /mnt/c/Users/Work/Desktop/Lab07/SE3082-2026-Sem02-Lab07/Exercise08$ make clean
rm -f ../Exercise01/sum_bcast ../Exercise02/sum_scatter ../Exercise03/sum_gather ../Exercise04/sum_reduce ../Exercise05/sum_allreduce ../Exercise06/sum_scan
work@LEGION: /mnt/c/Users/Work/Desktop/Lab07/SE3082-2026-Sem02-Lab07/Exercise08$ make all
mpicc -O3 -Wall -o ../Exercise01/sum_bcast ../Exercise01/sum_bcast.c
mpicc -O3 -Wall -o ../Exercise02/sum_scatter ../Exercise02/sum_scatter.c
mpicc -O3 -Wall -o ../Exercise03/sum_gather ../Exercise03/sum_gather.c
mpicc -O3 -Wall -o ../Exercise04/sum_reduce ../Exercise04/sum_reduce.c
mpicc -O3 -Wall -o ../Exercise05/sum_allreduce ../Exercise05/sum_allreduce.c
mpicc -O3 -Wall -o ../Exercise06/sum_scan ../Exercise06/sum_scan.c
work@LEGION: /mnt/c/Users/Work/Desktop/Lab07/SE3082-2026-Sem02-Lab07/Exercise08$ make run-local NP=16

=====
Running all exercises locally (np=16)
=====

>>> Running ../Exercise01/sum_bcast <<<
Root filled array with values 1 to 1000000
Rank 0: summed indices [0, 62500) => local_sum = 1953156250
Rank 8: summed indices [500000, 562500) => local_sum = 33203156250
Rank 4: summed indices [250000, 312500) => local_sum = 17578156250
Rank 12: summed indices [750000, 812500) => local_sum = 48828156250
Rank 2: summed indices [125000, 187500) => local_sum = 9765656250
Rank 10: summed indices [625000, 687500) => local_sum = 41015656250
Rank 6: summed indices [375000, 437500) => local_sum = 25390656250
Rank 14: summed indices [875000, 937500) => local_sum = 56640656250
Rank 15: summed indices [937500, 1000000) => local_sum = 60546906250
Rank 7: summed indices [437500, 500000) => local_sum = 2929606250
Rank 11: summed indices [687500, 750000) => local_sum = 44921006250
Rank 3: summed indices [187500, 250000) => local_sum = 13671906250
Rank 9: summed indices [562500, 625000) => local_sum = 37109406250
Rank 1: summed indices [62500, 125000) => local_sum = 5859406250
Rank 13: summed indices [812500, 875000) => local_sum = 52734406250

[Bcast] Total sum = 500000500000
[Bcast] Expected = 500000500000
[Bcast] Correct? = YES
[Bcast] Time = 0.044773 sec
Rank 5: summed indices [312500, 375000) => local_sum = 21484406250

>>> Running ../Exercise02/sum_scatter <<<
Root filled array with values 1 to 1000000
Rank 1: summed 62500 elements => local_sum = 5859406250
Rank 4: summed 62500 elements => local_sum = 17578156250
```